Analyse des sentiments en arabe avec AraBERT

Comprendre les données, le modèle, l'entraînement et l'évaluation, avec des illustrations.



Figure 1. Illustration conceptuelle générée par IA. Les couches représentées ne décrivent pas l'architecture exacte du modèle.

2	42	3
classes de sentiment	cellules du notebook	époques en mode complet

Dépôt du projet

github.com/yassinelaouni/arabert-sentiment-analysis

23 OCTOBRE 2026 / GUIDE EN FRANÇAIS

01 / LE PRINCIPE

Comprendre AraBERT

Un modèle de langue préentraîné devient un classifieur en apprenant sur des exemples annotés. Le notebook explique cette adaptation cellule par cellule. [1, 4]

Préentraînement

AraBERT apprend des régularités de l'arabe sur de grands corpus, notamment en prédisant des mots masqués. Il acquiert ainsi des représentations réutilisables. [4]

Fine-tuning

On ajoute une tête de classification à deux sorties, puis on entraîne le réseau sur des tweets annotés. Les poids de l'encodeur sont aussi ajustés pour apprendre le sentiment. [1]

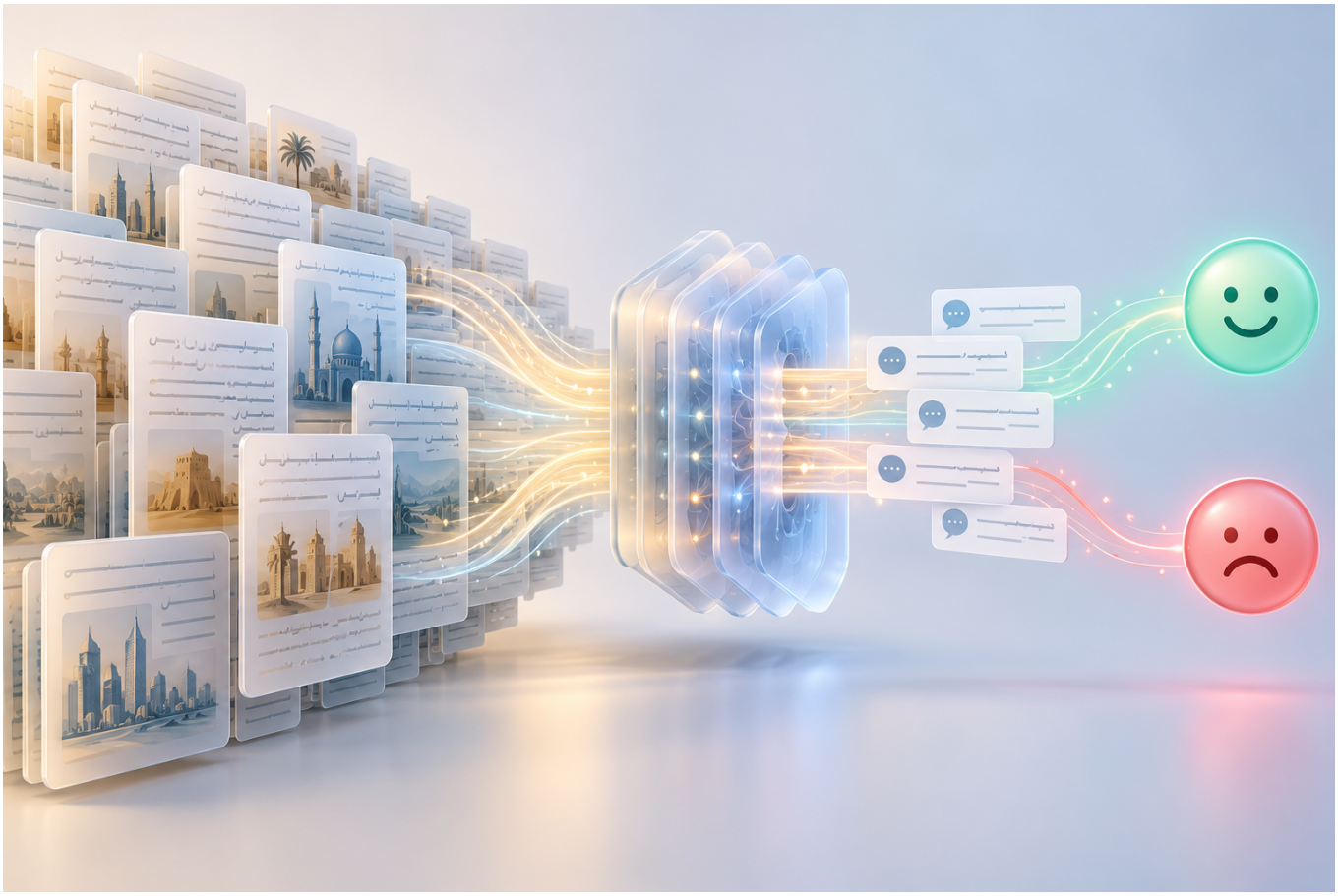


Figure 2. Illustration générée par IA : les connaissances linguistiques sont adaptées à la classification des sentiments. Le schéma est conceptuel.

Modèle choisi : aubmindlab/bert-base-arabertv02. Cette variante v0.2 n'utilise pas de segmentation Farasa ; celle-ci est donc désactivée. [4, 5]

02 / LES DONNÉES

Le corpus réellement utilisé

L'archive Kaggle a été téléchargée et ses quatre TSV ont été inspectés localement. Le découpage officiel entraînement/test est conservé. [1, 2]

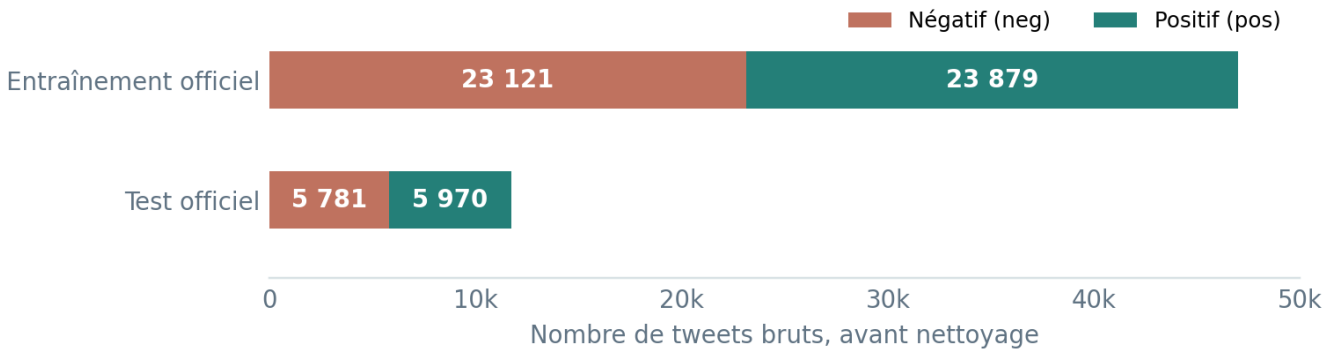
58 751
tweets bruts

47 000
lignes du train officiel

11 751
lignes du test officiel

Fichier original	Label	Lignes
`train_Arabic_tweets_negative_20190413.tsv`	neg	23 121
`train_Arabic_tweets_positive_20190413.tsv`	pos	23 879
`test_Arabic_tweets_negative_20190413.tsv`	neg	5 781
`test_Arabic_tweets_positive_20190413.tsv`	pos	5 970

Format : UTF-8, tabulations, sans en-tête ; deux colonnes : label, puis tweet. **Correspondance :** neg = 0 = négatif ; pos = 1 = positif. Aucune classe neutre.



Effectifs mesurés dans les fichiers d'origine, avant suppression des doublons et des labels contradictoires.

L'archive contient aussi des fichiers texte individuels représentant le même corpus : ils ne sont pas chargés avec les TSV. L'auteur décrit une collecte fondée sur les emojis, susceptible d'introduire des erreurs d'annotation. [3]

03 / PRÉPARATION DES DONNÉES

Préparer sans fuite de données

Des doublons peuvent gonfler artificiellement les scores. Le notebook compare les textes normalisés, puis les séquences exactes de tokens reçues par le modèle. [1]

1. **Vérifier les données d'entrée**
   Inspecter les fichiers, labels et encodages. Retirer les textes manquants ou vides, y compris ceux qui deviennent vides après prétraitement.

2. **Appliquer le prétraitement officiel**
   Conserver négations, emojis et répétitions expressives. Ne pas retirer les mots outils ni appliquer de racinisation. Farasa est désactivé pour v0.2.

3. **Donner la priorité au test officiel**
   Retirer du train les textes présents dans le test, sans consulter ses labels. Séparer ensuite le développement nettoyé en train/validation stratifiés.

4. **Traiter les groupes de doublons**
   Dans chaque ensemble, exclure les groupes aux labels contradictoires. Pour un texte répété avec un label cohérent, conserver une seule copie.

5. **Vérifier aussi les entrées du modèle**
   Comparer les séquences après troncature et gestion des tokens inconnus. Éliminer les recouvrements avec priorité au test, puis à la validation, puis au train.

14 592

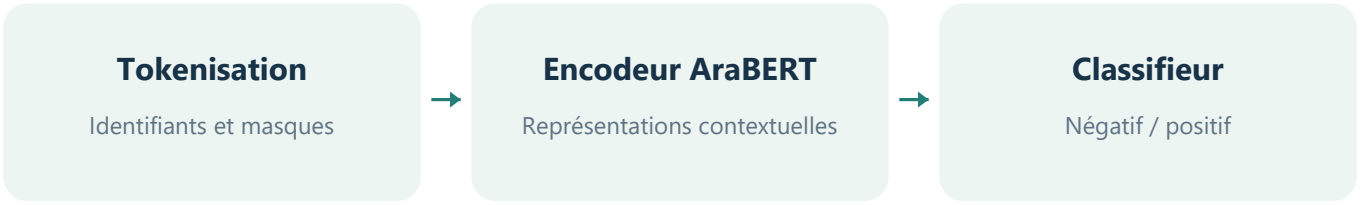
Lignes retirées du train car leur texte normalisé figurait aussi dans le test officiel. Ce constat provient de la préparation locale.

Ce nombre décrit la qualité des données ; il ne mesure pas les performances du modèle. Le test officiel est nettoyé et ne rejoint jamais l'entraînement. Les contrôles de doublons exacts ne détectent pas toutes les paraphrases.

04 / FINE-TUNING

Entraîner le réseau complet

PyTorch met à jour les poids. Le Trainer de Hugging Face gère les lots, la validation, les sauvegardes et le choix du meilleur modèle. [1, 6]



Flux simplifié. AutoModelForSequenceClassification charge AraBERT et ajoute la tête de classification. Tous les paramètres sont entraînables.

Paramètre	Configuration du notebook
Taux d'apprentissage	2e-5 (0,00002)
Époques	3 en mode complet ; 1 en mode rapide
Taille des lots sur GPU	8 si moins de 20 Gio ; sinon 16
Décroissance des poids	0,01
Accumulation des gradients	Lot effectif visé : au moins 16 exemples
Planification	Décroissance linéaire, préchauffage de 10 %
Choix de la sauvegarde	Meilleur F1 macro de validation par époque

Choisir la longueur à partir du train

La longueur retenue correspond au 95e percentile des longueurs tokenisées du train, arrondi au multiple de 8 supérieur et plafonné à 256 et à la limite du modèle. Le padding dynamique complète chaque lot selon ses besoins.

Adapter l'exécution au matériel

BF16 ou FP16 est activé uniquement si le GPU CUDA le permet et si le test réussit. CPU et MPS utilisent float32. Le recalcul des activations et l'accumulation limitent la mémoire. Le suivi externe est désactivé.

05 / ÉVALUATION

Évaluer avec rigueur

Choisir le modèle sur la validation. Évaluer le test seulement après ce choix, puis examiner les scores par classe et les erreurs individuelles. [1]

Exactitude (accuracy)

Proportion des exemples du test auxquels le modèle attribue la bonne classe.

Précision

Parmi les tweets attribués à une classe, part des prédictions correctes. La précision macro moyenne les deux classes.

Rappel

Parmi les tweets d'une classe réelle, part des exemples retrouvés. Le rappel macro moyenne les deux classes.

F1 macro

Moyenne des F1 des deux classes, calculés chacun à partir de la précision et du rappel. Ce critère sélectionne le modèle.

Sorties du notebook : courbes de perte train/validation, exactitude, précision/rappel/F1 macro, F1 par classe, rapport de classification, matrice de confusion et tableau d'erreurs avec probabilités.

Vérifié sur CPU

- Inspection et prétraitement de 58 751 lignes réelles.
- Préparation et séparation de 256 / 128 / 128 exemples vérifiées.
- Un pas d'optimisation : 4 exemples de train, 4 de validation, 4 de test.
- Évaluation, export et rechargement ; probabilités cohérentes à 1e-4 près.

Reste à exécuter

- L'entraînement rapide standard sur 256 exemples.
- L'entraînement complet sur trois époques du corpus nettoyé.
- L'exécution sur GPU CUDA avec précision mixte.

Aucun score sur le corpus complet n'est annoncé. Le test technique à quatre exemples vérifie l'exécution, sans mesurer la généralisation. Vérifications réalisées le 15 octobre 2026 ; guide préparé le 23 octobre 2026.

06 / UTILISER LE PROJET

Exécuter, puis prédire

Vérifier d'abord le mode rapide, puis redémarrer le noyau pour l'expérience complète. Cette pile de dépendances utilise Python 3.10 à 3.12. [1]

1. Créer un environnement Python local (Windows)

```
git clone https://github.com/yassinelaouni/arabert-sentiment-analysis.git
cd arabert-sentiment-analysis
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m ipykernel install --user --name arabert
.\.venv\Scripts\python.exe -m jupyterlab arabert_sentiment_analysis.ipynb
```

2. Fournir les données

Placer les quatre TSV originaux dans **data/** ou régler **DATA_DIR**. Sur Kaggle, joindre le dataset et activer Internet ; sur Colab, extraire l'archive ou utiliser un chemin Drive. Les identifiants Kaggle restent hors du notebook.

3. Exécuter les cellules dans l'ordre

Choisir le noyau **arabert**. Conserver **QUICK_MODE=True** pour commencer. Pour l'entraînement complet, mettre **QUICK_MODE=False**, redémarrer le noyau et tout rejouer. Redémarrer aussi après installation si des bibliothèques étaient déjà importées.

4. Prédire après l'entraînement

```
prediction = predict_sentiment(text)
print(prediction)
```

La fonction renvoie le sentiment et deux probabilités softmax. Les dernières cellules sauvegardent puis rechargent le modèle, le tokenizer et les mêmes réglages d'inférence.

الخدمة رائعة وأنا سعيد	لا أحب هذه الخدمة
Le service est excellent et je suis heureux.	Je n'aime pas ce service.

Exemples d'entrée uniquement : aucune prédiction ni probabilité n'est inventée.

07 / PERSPECTIVES

Limites et perspectives

Le fonctionnement du code est un point de départ. Le modèle final doit être évalué indépendamment sur plusieurs dialectes, thèmes et conditions d'annotation.

Sarcasme et contexte

L'ironie peut inverser le sens apparent d'un tweet. Son interprétation dépend parfois d'un contexte absent du message.

Couverture des dialectes

Darija marocaine, Arabizi et mélange de langues peuvent être moins bien reconnus. Ajouter des exemples annotés du public visé.

Biais d'annotation

Les labels fondés sur les emojis peuvent favoriser des raccourcis. Auditer les annotations et tester la robustesse avec et sans emojis. [3]

Probabilités incertaines

Les scores softmax ne sont pas calibrés. Le modèle binaire attribue aussi une polarité aux sentiments neutres ou mixtes.

Améliorations réalistes : annoter un test en darija, rechercher les quasi-doublons, comparer un AraBERT adapté aux tweets sur la validation, essayer plusieurs graines, calibrer les probabilités et ajouter une classe neutre avec des données adaptées.

Sources et ressources du projet

[1] Dépôt du projet et notes de vérification
https://github.com/yassinelaouni/arabert-sentiment-analysis

[2] Kaggle : Arabic Sentiment Twitter Corpus
https://www.kaggle.com/datasets/mksaad/arabic-sentiment-twitter-corpus

[3] Auteur du corpus : collecte et annotation
https://github.com/motazsaad/arabic-sentiment-analysis

[4] Fiche du modèle AraBERT v0.2
https://huggingface.co/aubmindlab/bert-base-arabertv02

[5] Prétraitement officiel d'AraBERT
https://github.com/aub-mind/arabert/blob/master/preprocess.py

[6] Hugging Face Trainer, API utilisée : v4.51.3
https://huggingface.co/docs/transformers/v4.51.3/en/main_classes/trainer

Figures 1-2 : images créées avec l'outil de génération d'images de ChatGPT. Ces illustrations conceptuelles ne représentent ni des résultats mesurés ni l'architecture exacte de BERT. Les prompts et images d'origine accompagnent le guide. Les effectifs et constats de préparation proviennent des fichiers réels.